qcode

User Manual

A terminal-first AI coding assistant.
This guide shows how to use qcode day to day.

Version dev - November 07, 2026
Usage guide for people using qcode.
No setup internals or code architecture inside.

In this manual: install once, tour the screen (including pasted blocks and the execution checklist), try practical use cases, clarifying questions, switch models and providers (including ChatGPT login), use agents with working directories and allowed-model roster, plan, todo, skills, memory, tools with approvals, sessions, browser control, and settings.

How to use this manual

Read chapters 1-3 to start. Keep chapters 4 and 15 nearby as a command reference. The Use cases pages after chapter 3 offer complete example workflows. Chapters 5-12 are task guides you can read when you need them: models, agents with working directories and allowed-model roster, planning, skills, memory, tools with approvals and web, sessions, and phone/browser control.

- Words in `code style` are things you type, for example `/model` or `qcode --demo`.
- Figures are illustrative example screens. Your colors and sizes may differ slightly.
- The prompt line shows `>` when idle, `(Plan)>` in Plan mode, `(Skill plan)>` in Skill Plan mode, and `(Steer)>` while busy.
- Esc goes back one level. Ctrl+C closes a command view; agents continue working. In the composer, Ctrl+C cancels active work.

Tip

Run qcode --demo for the interactive tour with automatic queued prompts. Add a quoted prompt for a one-shot text demo. Both use mocked tools and leave your workspace untouched.

1. Getting started

Install and first run

qcode is one program you run in your terminal. Build it once, then start it in any project folder.

```
# Build from source (needs Go 1.22+)
make build
./bin/qcode
```

The remaining shell examples assume qcode is on your PATH. From this source checkout, use ./bin/qcode instead. make build creates the binary but does not install it.

Pick the assistant you want to use. The two common choices are a local model (Ollama) or a hosted model (OpenAI or OpenCode Go).

```
# Local model (default provider)
ollama pull qwen2.5-coder:7b
qcode --model qwen2.5-coder:7b

# Hosted model
export OPENAI_API_KEY=...
qcode --provider openai --model gpt-5

# No setup interactive tour
qcode --demo
```

Tip

If you only want to learn the keys and screens, start with --demo. It runs a scripted tour with fake tools and queued prompts.

Update

Use the shell command below to install the latest release. It is a qcode CLI command, not a slash command.

```
qcode update
qcode update --arch arm64
qcode update --pre
qcode update --help
```

Without --pre only stable releases are considered; --pre also considers pre-releases such as release candidates. The command installs the latest release for the current operating system and chosen architecture.

Startup flags and environment

qcode --help lists every flag supported by your installed binary. The most useful launch choices are workspace, model, thinking, step limit, context accounting, compaction, sandbox, networking, and one-shot output. Put flags before any one-shot prompt; option parsing stops at the first positional word.

```
qcode --cwd /path/to/project
qcode --provider ollama --model qwen3:8b --thinking high "reply with OK"
qcode --max-steps 64 "explain this repository"
qcode --context-window 32768 "explain this repository"
qcode --disable-auto-compact
qcode --sandbox --sandbox-command-path ~/.local/bin
qcode --danger-skip-tls-verify "run the tests"
qcode --agent-timeout 90s "explain this repository"
qcode --json-events "run the tests" 2>events.jsonl
qcode --list-providers
qcode --provider openai --update-model-meta
qcode --version
```

- --cwd DIRECTORY selects the workspace for tools, skills, and saved-session lookup without changing your shell directory (default `.`).
- --provider, --model, --thinking, --base-url, and --api-key select the assistant. Flags beat QCODE_PROVIDER, QCODE_MODEL, QCODE_THINKING, QCODE_BASE_URL, QCODE_API_KEY (or OPENAI_API_KEY fallback), which beat config.toml, which beats built-ins. There is no --interactive or --theme flag: set interactive and theme in config.toml or use /interactive and /theme.
- --max-steps N sets the per-request model-turn limit (default 32); /maxsteps shows or changes it later. --agent-timeout DURATION sets how long main waits for a batch of agent consultations including queue time (default 5m, for example 90s).
- --context-window N overrides terminal-agent context accounting for an unlisted model (0 means automatic); switching models clears it. --disable-auto-compact disables automatic summaries in terminal sessions; /compact stays available. One-shot runs use provider capacity and the built-in 80% threshold.
- --sandbox requests Linux bubblewrap isolation; --sandbox=false overrides config. Repeat --sandbox-command-path for each trusted executable directory (it replaces sandbox_command_paths). Check the startup notice to confirm isolation is active.
- --danger-skip-tls-verify disables certificate verification for qcode HTTP clients and supplies insecure-TLS settings to supported shell clients.
- --json-events emits JSON Lines trace, activity, and token-usage events and never waits for questions; answers stay on stdout while activity goes to stderr. --demo runs the offline scripted tour with mocked tools and skips configuration and saved sessions. --demo with a quoted prompt runs one text demo.
- --list-providers prints built-in provider names and exits. --update-model-meta refreshes cached thinking choices for --provider openai or opencode-go and exits; it does not update the binary or context-window catalog.

2. Touring the terminal

qcode fills the terminal but keeps your normal scrollback. Tabs sit at the top, with the latest received prompt highlighted directly beneath them. The transcript scrolls in the middle; pending work, the editable composer, task indicator, and status bar stay at the bottom.

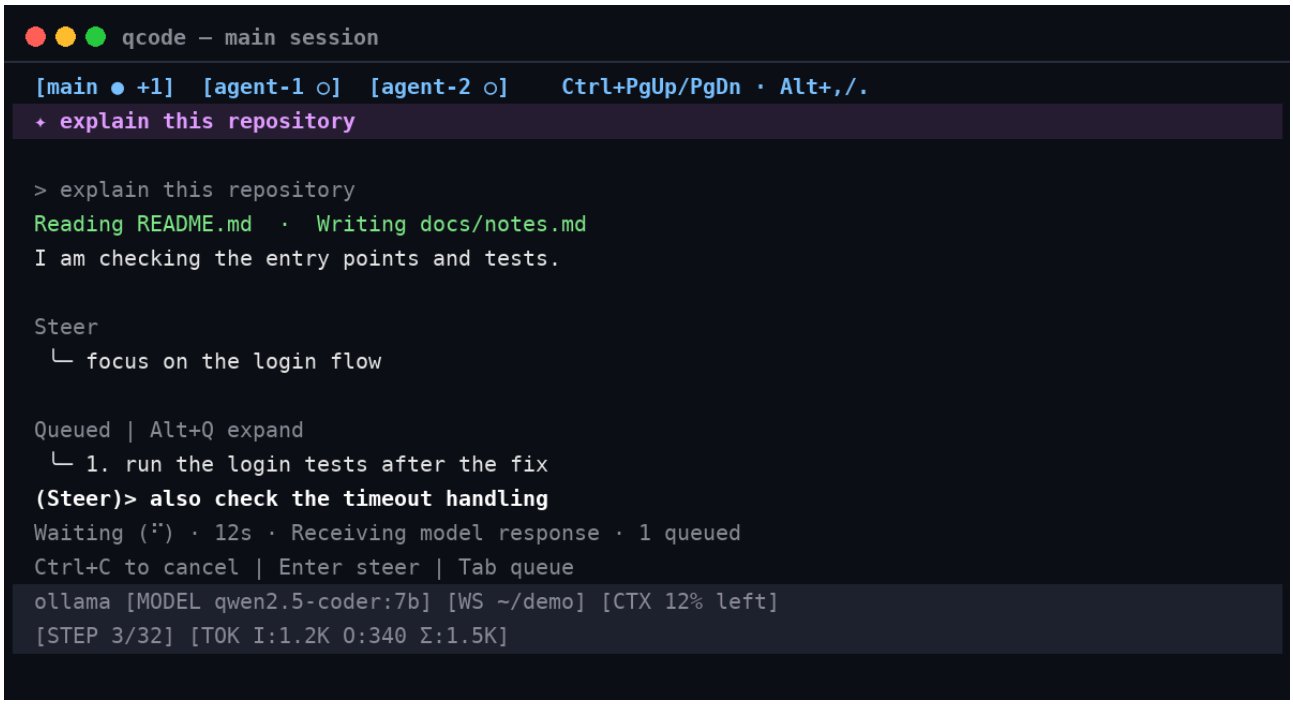


Figure 1: Main screen. The highlighted prompt stays below the tabs; pending steering and queued tasks appear above the composer.

What you see

- Tabs: `main` is always there. Extra agents appear as new tabs (up to 20 total).
- Latest prompt: a star marks up to three highlighted lines beneath the tabs. It changes when a task starts or steering is delivered. Pending input does not change it; completion keeps it, /clear and /new clear it, and /resume restores it. Small windows show fewer lines.
- Transcript: streamed answers with Markdown, tables, and tool activity like `Reading ...` or `Writing ...`. One blank separator row sits between each pair of visible regions (pinned prompt, transcript, pending prompts, suggestions, Steer/Queued blocks, composer); very small terminals keep minimum output visible and omit separators that would hide content.
- Task indicator: Waiting for model response, Receiving model thinking, or Receiving model response, with elapsed time for that operation. The timer resets when the phase changes. Tools appear in the transcript; questions show Waiting for input.
- Pending area: shows the current steer separately from FIFO tasks.
- Todo panel: a live `Todo done/total` checklist sits above the steer/queue area while multi-step work is incomplete (`[ ]` todo, half-circle doing, `[x]` done). Alt+T expands it; Esc collapses; /todo show reviews it any time.
- Prompt: `>` idle, `(Plan)>` planning, `(Skill plan)>` skill design, `(Steer)>` working. You can keep typing while work runs.

- Status bar: provider, MODEL, WS folder, CTX percent left, STEP n/max, and TOK input/output plus total when it fits. THINK appears when set, MODE shows PLAN, SKILL PLAN, or INTERACTIVE, and REMOTE while browser control runs. Narrow windows shorten the workspace path first, then wrap to a second left-aligned row before dropping low-priority segments (tok first). Hide parts with /statusline.
- Pastes: long pastes stay together as one `[paste N characters]` block (100+ characters or more than 3 lines; one trailing newline does not count as a line). Short pastes stay inline and editable. Pasting never submits: press Enter to submit the full draft. Type before or after a collapsed block; Left/Right moves across it as a whole, Backspace/Delete removes the whole block.

Keys you will use daily

- Type `/` to see up to five matching commands. Type more to filter, Tab to complete the first match, Esc to close.
- Left/Right edits input. At the bottom of output, Home/End move to input ends. Ctrl+Left/Right (or Alt+Left/Right, Alt+B/F) jumps by word; Ctrl+W deletes a word; Ctrl+A/E moves to input ends.
- PageUp/PageDown scrolls history with two overlapping rows of context, even while the agent works. Scrolling pauses the live view; PageDown to the bottom resumes it. While paused, Up/Down moves one row for fine reading; it edits the draft again once the live view resumes.
- Press Up/Down at an empty prompt to recall earlier input and return to your unfinished draft.
- When the view is above the bottom, Home/End jump to oldest/latest output. At the bottom they edit the prompt again. Browser text fields keep editing keys.
- Alt+Q opens pending work. Up/Down selects, PgUp/PgDn scrolls, Delete removes, and Esc closes.
- Ctrl+C in the composer cancels the running prompt; the next queued prompt can start. In a command view or picker, it closes the view while agents keep working; it also cancels a running /bash command without exiting qcode.
- NO_COLOR=1 disables colors; QCODE_ASCII=1 uses ASCII glyphs.

3. Asking, queueing, and history

Type a request and press Enter. While one request runs, you can type the next one. Enter steers the current task; Tab queues a separate task. Steering waits for a response or tool, replaces pending steering, and withdraws unanswered interactions.

The pending panel separates Steer from Queued tasks with tree connectors. Use Alt+Q to expand it, Up/Down to select, Delete to remove an undelivered item, and Esc or Alt+Q to return to typing. Once steering starts replanning, it cannot be removed. Completed tool changes remain in place when you steer or cancel.

- One prompt runs at a time per agent. Other agents are not blocked.
- The tab and task line show the queued task count.
- The newest pending steer replaces earlier pending steering. If the task ends before your steer is accepted, qcode rejects it and keeps the draft. Cancellation or failure drops undelivered steers; queued tasks still continue.
- Tab still completes matching slash commands. During manual /compact, Enter cannot steer and keeps your draft; Tab can queue a separate task.
- Use `/history` to find an old prompt. Pick one to re-read its final answer.
- Use `/new` to clear the active tab's conversation, latest prompt, usage, and plans; it restores web tools to on outside the sandbox and off inside it, removes added directory grants, and resets the working directory to launch. Models, selected skills, and the questions toggle stay.
- Use `/compact` to summarize old context, or `/compact show` to inspect the compacted context (system + summary + recent tail) in a history-style debug view. Automatic compaction starts at 80% use when capacity is known; change auto_compact_threshold or disable_auto_compact in config.toml. During manual /compact, Enter cannot steer and keeps your draft.

Tip

Example: submit `fix the login timeout` with Enter. While it runs, submit `preserve the existing API` with Enter to steer it, or `run the login tests after the fix` with Tab to queue a separate task.

When qcode asks you back (/interactive)

Normal work can pause to ask one focused question when an ambiguity would otherwise need several broad searches. Turn this on per agent with `/interactive on`, off with `/interactive off`, or check the setting with `/interactive`. It is off by default. Add `interactive = true` to config.toml to enable it for new terminal sessions.

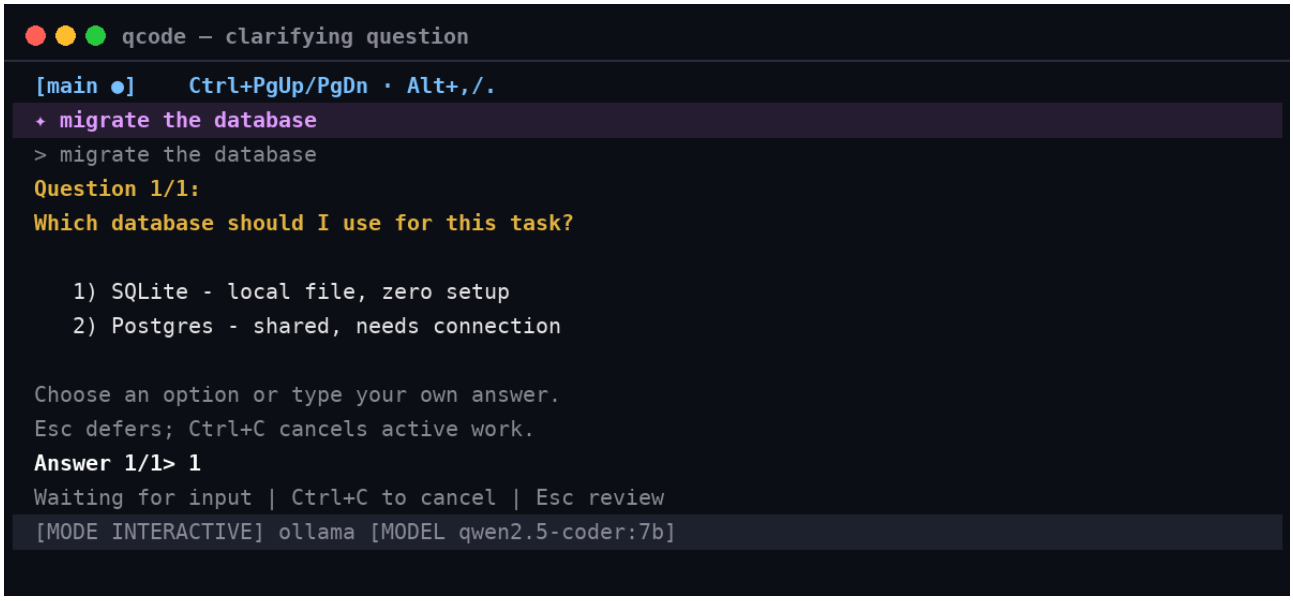


Figure 2: A clarifying question. Type an option number or your own answer. At most 3 questions per prompt.

- One question at a time, with suggested choices or your own custom answer. At most 3 distinct questions per submitted prompt.
- Your answer becomes context for that agent and is included in saved sessions. It is not added to global learning automatically; use /learn to retain a reusable preference.
- The status bar shows [MODE INTERACTIVE] while enabled ([MODE INT] when narrow). Your half-typed draft is saved and restored around the question.
- /interactive on|off saves the default for future terminal sessions from any agent tab. Each existing or resumed tab keeps its own setting; demo changes stay session-only.
- Esc defers a question and opens the composer; Esc returns to it. A steer withdraws unanswered interactions. Ctrl+C cancels active work. The browser can also answer.
- One-shot prompts, piped input, non-terminal runs, and --json-events never wait; qcode just proceeds with available context.

Tip

Turn it on when tasks are ambiguous (which database, which scope). Leave it off for strict hands-off runs.

Use cases: understand and fix

These are example requests, not built-in commands. Replace paths, commands, and symptoms with those from your project. Review the resulting files and command output before accepting a change.

Understand an unfamiliar repository

Goal: find the entry points and learn how to run the project before making changes.

1. Start qcode in the project folder and submit the example below.
2. Ask a follow-up about the part you need to change. Use /history to revisit the explanation.
3. Check the cited files and confirm the suggested commands against the project's README.

```
Explain this repository without changing files. Identify the
entry points, how to run it, and how to run its tests. Cite the
files you used, and list anything you could not confirm.
```

Result to look for: a file-grounded starting guide with clear run and test commands, plus open questions.

Fix a bug and queue the next step

Goal: reproduce a specific failure, make a focused fix, and check the result.

1. Use /interactive on if expected behavior needs clarification; then describe the failure and a reproduction command.
2. While the fix runs, press Tab to queue the follow-up below. It waits in the active agent's FIFO queue.
3. Review the diff with /diff and inspect test output. Ask for an explanation if the checks fail.

```
Fix the login timeout: retry once after a temporary network
failure, but never retry invalid credentials. Reproduce it
with the login tests before changing the handler.

# Press Tab to queue while the first request is running:
Run the login tests after the fix and summarize the results.
```

Result to look for: a small diff and observed test results, with any remaining failure stated explicitly.

Use cases: plan and divide work

Plan a change before editing

Goal: settle scope and review the approach before implementation.

1. On an idle agent, enter /plan and describe the change below.
2. Answer design questions, then use /plan show to review the proposal. Request revisions if needed.
3. Use /plan act only when ready to implement. Review the diff and the planned checks afterward; /plan off leaves without implementing.

```
/plan
Plan pagination for the search results. Inspect the current
API and UI, identify compatibility concerns, and propose
acceptance checks before changing files.
/plan show
/plan act
```

Result to look for: an agreed plan followed by changes that meet its acceptance checks.

Use independent agents for implementation and investigation

Goal: keep a focused implementation moving while another tab investigates a separate question.

1. Give main the implementation task. Use /agent new to pick a model for a helper; rename its displayed ID with /agent rename <id> review-tests.
2. In the helper tab, ask for a read-only review of relevant tests and missing edge cases. Switch tabs with Ctrl+PageUp/PageDown.
3. Bring the helper's findings back to main, request the needed checks, and review the combined result.

```
# Main prompt:
Add the agreed pagination behavior to the search endpoint.

# Helper prompt:
Read the search tests without changing files. Identify
missing pagination edge cases and report the file locations.
```

Tip

Agents share the workspace. Give them distinct responsibilities; avoid asking two tabs to edit the same files at once. Each tab has its own history and queue.

Use cases: reuse and continue

Turn a repeated review into a skill

Goal: make a repeatable checklist available in future work.

1. Start /skillplan with the example below and answer questions about scope, inputs, and validation.
2. Review with /skillplan show; request edits until the draft describes your actual workflow.
3. Use /skillplan create to write the reviewed skill, then /skill to enable it before the next review.

```
/skillplan Create a database migration review checklist.
Check rollback behavior, existing data compatibility,
and validation commands. Report risks with file references.
/skillplan show
/skillplan create
/skill
```

Result to look for: an enabled, reusable skill whose instructions and completion checks you have reviewed.

Resume work and share its outcome

Goal: continue a saved conversation and produce a readable handoff.

1. Return to the same project folder, start qcode, and use /resume to choose the saved session.
2. Use /history to find the last completed answer. Ask for current status and remaining work before continuing; interrupted tools are not rerun automatically.
3. Use /export pretty for a shareable HTML conversation, or /export raw when the handoff needs detailed activity.

```
/resume
Summarize the completed changes and remaining work. Verify
the current files before continuing the unfinished task.
/export pretty
```

Tip

To continue from a phone on a trusted network, use /remote and redeem the QR link. The browser controls the same live session; its /export downloads the HTML. Close the connection when finished.

4. Slash commands at a glance

Type `/help` to list commands or `/help <name>` for one command. The leading `/` is optional in the name. Agent, skill, learning, plan, remote, model, tool, and interactive commands are covered in their chapters below.

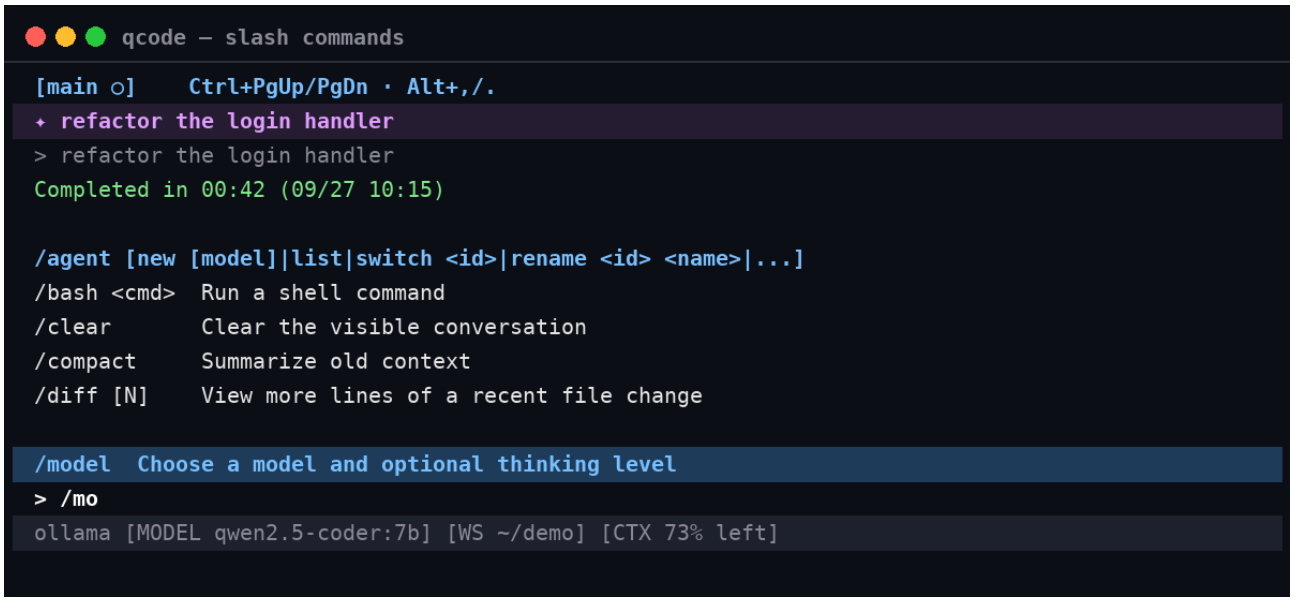


Figure 3: /help lists commands. Typing /mo filters the suggestions to /model; Tab completes the first match.

Command	What it does for you
/help	List commands or explain one command.
/bash <cmd>	Run a shell command in the active agent's environment.
/new	Start a fresh conversation for the active agent.
/resume [session-id]	Preview, rename, pin, delete, or continue saved sessions.
/clear	Clear the visible conversation and redraw the header.
/history	Browse completed prompts and responses.
/memo [text]	Save a scratchpad memo or manage memos; put-to-prompt fills input.
/diff [N]	View more lines of saved diff N; omit N for latest.
/verbose	Show or hide detailed action traces.
/maxsteps [N]	Show or change the model-turn limit.
/statusline	Show, hide, or reset status bar parts.
/theme	Preview, apply, and save a terminal color theme.
/compact [show]	Summarize old context; show inspects the compacted context.
/export	Export conversations or full transcript as HTML.
/provider [name]	Switch the active idle tab without restarting; terminal-only.
/todo show	Review the live execution checklist; Alt+T expands it.

Command	What it does for you
/agent ...	Create, list, switch, rename, cancel, or close agent tabs.
/skill [...]	Pick reusable skills; Space cycles off/lazy/always-load.
/plan ...	Enter, review, implement, or leave read-only planning.
/interactive ...	Toggle inline clarifying questions for normal work.
/learn ...	Save, list, forget, or compact reusable preferences.
/skillplan ...	Guided review-first skill creation.
/remote	Control the live session from a browser.
/quit, /exit	Save the session and exit qcode.

Compacted context inspector

After a manual or automatic compaction, the terminal hints `Type /compact show to inspect the compacted context.` Use `/compact show` to open a history-style debug view with the compacted conversation (system prompt + summary + recent tail). It is read-only and helps you check what was kept before continuing.

Diff previews

When qcode writes or edits a file you see a short numbered 10-line preview with added and removed lines. Use `/diff` to expand the latest saved diff or `/diff N` to expand saved diff N, up to the 200-line safety limit. The model still receives plain text.

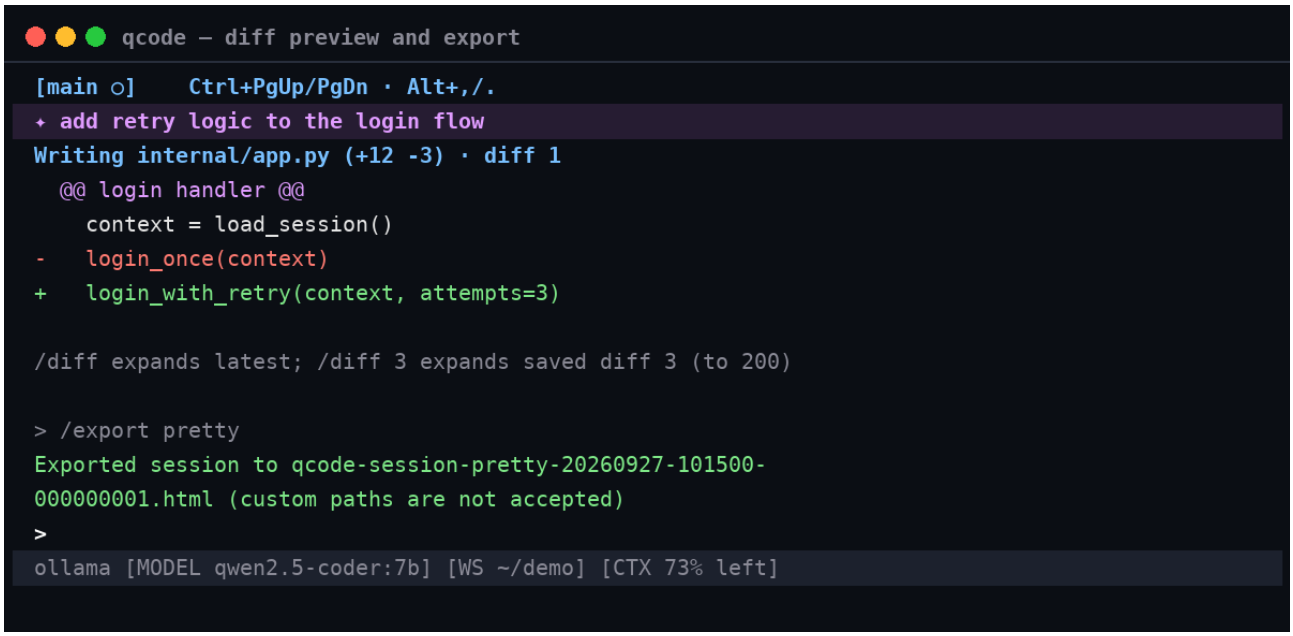


Figure 4: Numbered change preview plus /export pretty saving a readable HTML copy.

Exports

- `/export` or `/export pretty`: one tab per agent, prompt cards that reveal answers. Best for sharing and reading.
- `/export raw`: full styled transcript including tool activity, thinking, errors, and diffs.
- In a pretty export, Show detailed tool events reveals concise activity beside each exchange; it starts unchecked.
- Files are saved as `qcode-session-<mode>-<timestamp>.html` with subsecond precision. Custom output paths are not accepted.
- Terminal exports write in the workspace; browser exports download the same HTML without writing a host file.

5. Choosing models and providers

Use `/model` to switch the assistant for the current tab. The list shows model names from your current provider only, with no provider suffix. Type to search, move with Up/Down and PgUp/PgDn, Enter to pick. You can also run `/model <id> [thinking]` directly. Some models ask a second question for thinking level; Esc returns to the model list and Ctrl+C cancels.



Figure 5: /model picker. Model names only; the list comes from the current provider. An optional thinking level follows when the model offers one (for example off/low/medium/high/max; other models offer on, none, minimal, or xhigh). Choices depend on the model.

Providers you can use

- Ollama (local default): pull a model first, then `qcode --model qwen2.5-coder:7b`. Thinking uses the model's native think setting.
- OpenAI-compatible: `export OPENAI_API_KEY=...` then `qcode --provider openai --model gpt-5`. Thinking uses reasoning effort where the model offers it.
- OpenCode Go: `export QCODE_API_KEY=...` then `qcode --provider opencode-go --model kimi-k3`. Options vary per model. Known models may offer off, on, none, minimal, low, medium, high, xhigh, or max; the set is model-specific. With no setting, qcode preserves the provider default and omits optional thinking fields.
- ChatGPT subscription: `qcode login chatgpt` connects Sign in with ChatGPT (browser approval, loopback callback, 0600 OAuth file), then `qcode --provider chatgpt --model gpt-5.6-codex`. Requests use the SIWC Responses endpoint with store:false; thinking levels minimal/low/medium/high/xhigh map to Responses reasoning effort.
- Use `/provider` (or `/provider openai`) to switch an idle tab without restarting; unconfigured providers are dimmed and cannot be chosen. Main-tab switches save the default; workers stay local. History is kept with a `/new` warning; browser provider commands are rejected. Keep separate endpoints/keys with [providers.<name>] connection profiles.
- Flags can also come from QCODE_PROVIDER, QCODE_MODEL, QCODE_THINKING, QCODE_BASE_URL, QCODE_API_KEY.

```
qcode --provider ollama --model qwen3:8b --thinking high "reply with OK"
qcode --provider opencode-go --model deepseek-v4-flash --thinking high "reply with OK"
qcode login chatgpt
qcode --provider chatgpt --model gpt-5.6-codex "explain this repository"
```

- Switching to a model from a different provider family warns before applying; the same-backend switch applies silently. A failed or unsupported explicit level fails before any model request.
- OpenAI and OpenCode Go read thinking choices from the public Models.dev catalog, cached under your cache directory (openai-model-meta-v1.json, opencode-go-model-meta-v1.json). Refresh with qcode --provider openai --update-model-meta or qcode --provider opencode-go --update-model-meta. A failed first fetch runs without selectable thinking; a failed refresh keeps the last valid cache. Custom --base-url endpoints do not load this metadata.
- Ollama discovers thinking support from local model metadata and sends it through its native think field (GPT-OSS supports low/medium/high and cannot disable thinking that way). Unknown model IDs stay usable but omit optional thinking fields.
- Vision-capable models can inspect PNG, JPEG, WEBP, and GIF files inside the workspace (20 MiB each) when you name the image; the agent loads it with view_image through the active provider. The model and endpoint must support image input.

Tip

On `main`, /model and /maxsteps remember your choice. Other tabs, demo, one-shot, and piped runs keep changes for that session only.

One-shot and demo

```
qcode "explain this repository"
qcode --json-events "run the tests" 2>events.jsonl
printf '%s\n' 'explain this repository' | qcode
qcode --demo "show me how qcode works"
```

Put flags before one-shot prompts or piped text. Empty stdin fails. Answers use stdout; activity or diagnostics use stderr. One-shot and piped runs never wait for questions or directory approvals and do not save sessions. Configured skill autoloading and learning retrieval still apply. With --json-events, each completion also emits a usage event with provider, model, per-completion usage (null when unavailable), and session totals. Demo tools are mocked; /export still writes a file. --demo without a prompt starts the automatic interactive showcase.

6. Working with multiple agents

Use extra agents to do independent jobs at the same time, for example one writing code while another reads tests. Each tab keeps its own history, draft, queue, model, and tool choices.



Figure 6: /agent list. Enter switches to the highlighted agent.

Command	What it does for you
/agent	Create an agent (same as /agent new).
/agent new [model]	Create an agent, optionally with a model ID.
/agent list	Pick and switch with Up/Down + Enter.
/agent switch <id>	Jump directly to one agent.
/agent rename <id> <name>	Give an agent a new display name.
/agent cancel <id>	Stop that agent's current work.
/agent close <id>	Cancel work and close the tab after the run exits (--yes skips confirm).
/agent allow-models	Curate which models main may use for helpers.
/agent allow-models list	Show the current provider roster.
/agent allow-models clear	Empty only the current provider (no restriction).

- Switch with Ctrl+PageUp/PageDown (Alt+, and Alt+. also work).
- Ask the main agent to split work and it can coordinate helpers; you can also create tabs yourself with /agent.
- /agent and /agent new open the current provider's model picker. /agent new <model> takes a model ID, then /agent rename <id> <name> sets its display name.
- Queues are per agent (up to 16 waiting each). A slow helper never blocks `main`.

- Each agent has its own working directory, starting at the launch workspace (--cwd). Use the change_directory tool (ask for packages/server) to select another existing directory; relative paths resolve against that agent’s current directory and symlinks resolve to their canonical destination. The status WS segment updates during the task; other agents and qcode keep their own directories. Helpers created by main inherit its provider/endpoint, tools, skills, directory grants, working directory, and step limit; an omitted model inherits main’s model and an omitted thinking_level uses the provider default (never main’s level).
- Ask main to wait for helpers with a consultation deadline: --agent-timeout 90s (default 5m) is shared by the batch including queue time. Completed replies are kept; failed or expired consultations are reported and skipped. This applies to main’s consult_agents coordination, not ordinary prompts or background delegate_task work.
- Curate helper models with /agent allow-models: type to filter the current provider, Space cycles off -> smart -> balanced -> cheap -> off, Enter saves. Pick only model plus role (smart = hard reasoning, balanced = routine, cheap = trivial); there is no thinking picker. /agent allow-models list shows the roster, clear empties only that provider, and main-tab saves persist [[agents.allow_models]] to the user config (worker-tab edits stay session-only). Empty means no restriction.
- When a roster exists, main picks by hardness before creating a helper: hard uses smart with the high end of its thinking_levels, routine uses balanced with the default, trivial uses cheap with a low level or omitted. Main decides thinking_level at creation time from live capability data; an unsupported level fails creation with no child.
- When main delegates a task (create_agent with task or delegate_task), its current run ends after accepting the assignment and the helper continues in its tab. When the helper finishes, main receives an automatic callback with a bounded summary and reacts to it; burst completions coalesce into one pending callback. Use search_agent_work or consult_agents for details instead of polling list_agents. Full recorded findings remain searchable after closing a helper or using /new.
- Main can use close_agent with one agent_id in normal execution mode. Closure returns immediately without ending main's turn. Pending tasks, steering, and interactions are cancelled; a running helper shows closing until its run exits and still counts toward the agent limit. The selected closed tab returns to main. Closing helpers are excluded from saved resumable tabs; recorded work remains searchable and exportable.

7. Planning before coding

Plan mode investigates without changing files. Enter it while idle, let qcode study the code, answer any design questions, review the submitted plan, then build it.



Figure 7: Plan mode. The prompt shows (Plan) and the status bar shows [MODE PLAN] until you act or leave.

1. Type `/plan` to enter. The status bar shows PLAN.
2. Describe the task. Answer focused questions if qcode asks (you can type a custom answer).
3. Type `/plan show` to re-read the latest plan any time.
4. Type `/plan act` to build the latest plan, or `/plan off` to leave without building.

- Plan mode only advertises inspection tools (read, list, search, view_image, skills, and explicitly enabled web tools). Mutation, shell execution, extra directory grants, and sub-agent delegation are rejected at the tool boundary.
- Earlier plan revisions stay in the conversation, but only the latest submitted plan is used for the /plan act handoff.

Tip

`/new` clears the saved plan. Plan commands need an idle agent with no running or queued work.

Execution checklist (/todo)

For multi-step implementation the model keeps one execution checklist per agent tab with todo_add, todo_update, and todo_done. It is separate from /plan: plans are one-shot handoffs, while the todo list is updated while building. Incomplete lists persist across requests; a finished list is archived when observed and cleared when the next request starts. Sub-agents keep their own lists; /plan act keeps the todo list and /new clears both.

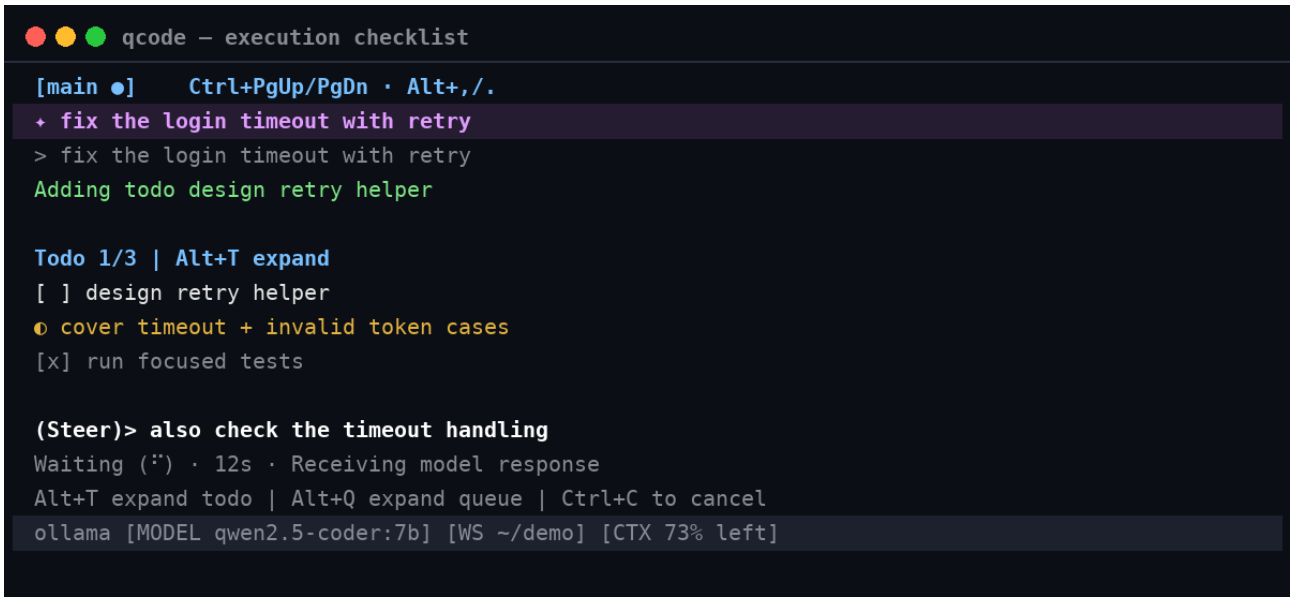


Figure 7b: Live Todo panel above the steer/queue area. Alt+T expands it; /todo show reviews the latest checklist.

- A live `Todo done/total` panel sits above the steer/queue area while work is incomplete ([ ] todo, half-circle doing, [x] done). Collapsed it shows up to three rows; Alt+T expands all rows and Esc collapses.
- Use `/todo show` to open the latest checklist in a scrollable view at any time. A finished list is archived once to the transcript and cleared when the next request starts.
- Titles are one line (20 items max); the model should only call todo tools when the list changes, not every turn. Todo tools are unavailable in Plan and Skill Plan modes.

8. Skills and SkillPlan

Using skills

Skills are reusable instruction bundles, for example a team review checklist. Turn them on per tab with `/skill`. Lazy skills initially add only names and summaries to context; the agent loads the full SKILL.md when needed. Always-loaded skills inject the full SKILL.md eagerly. You can also run `/skill name1,name2` directly, `/skill name!` for always-load, or `/skill none` to clear them.

- Type `/skill` to see names plus one-line summaries, sizes, and autoload badges. Type to filter, Up/Down or PgUp/PgDn to move, Space to cycle off -> lazy ([x]) -> always ([*]), Enter to apply, Esc to leave, Ctrl+C to cancel. The header shows lazy/always counts plus a stacked context bar (always vs catalog over base + always + catalog).
- Your account, workspace, and configured folders can each provide skills.
- New files appear in `/skill` without restarting.
- The source repository's docs/skills collection is optional. Add its absolute directory to [skills] paths in config.toml, or copy a chosen folder into a built-in root; release binaries do not bundle these files.
- Use [skills] autoload_paths (lazy) or [[skills.autoload]] with always_load = true to select every valid skill in a directory for new runs, including one-shot and piped prompts. /skill on main also saves [skills] selected/selected_always in the user config for new runs. Resumed tabs retain their saved selection, including always-load flags.
- The bundled collection includes bounded-verification (stop re-checking once no unresolved fact could change the answer), pdf-read (parse, chunk, retrieve, and reason over long PDFs), and skill-creator (turn a workflow brief into a focused SKILL.md). Enable any of them with /skill after adding the collection to [skills] paths.
- A skill can ship companion files (scripts, templates, data) beside SKILL.md. The agent is told the skill’s base directory and loads companions only when needed with read, view_image, or shell; loading a skill does not change the working directory or grant directory access.

Creating a skill safely

Use `/skillplan` alone or with a rough idea to turn it into a reviewed skill. Entering from another mode starts a fresh draft; `/skillplan <rough intention>` submits that idea immediately and prints that guided design started. The prompt shows (Skill plan)> and the status bar shows [MODE SKILL PLAN]. It asks one focused question at a time about when the skill applies, its workflow, inputs and outputs, constraints, validation, name, and scope, with follow-ups when an answer leaves a decision open. Questions offer numbered choices (a description-only answer is shown as the choice) and always accept your own custom answer.

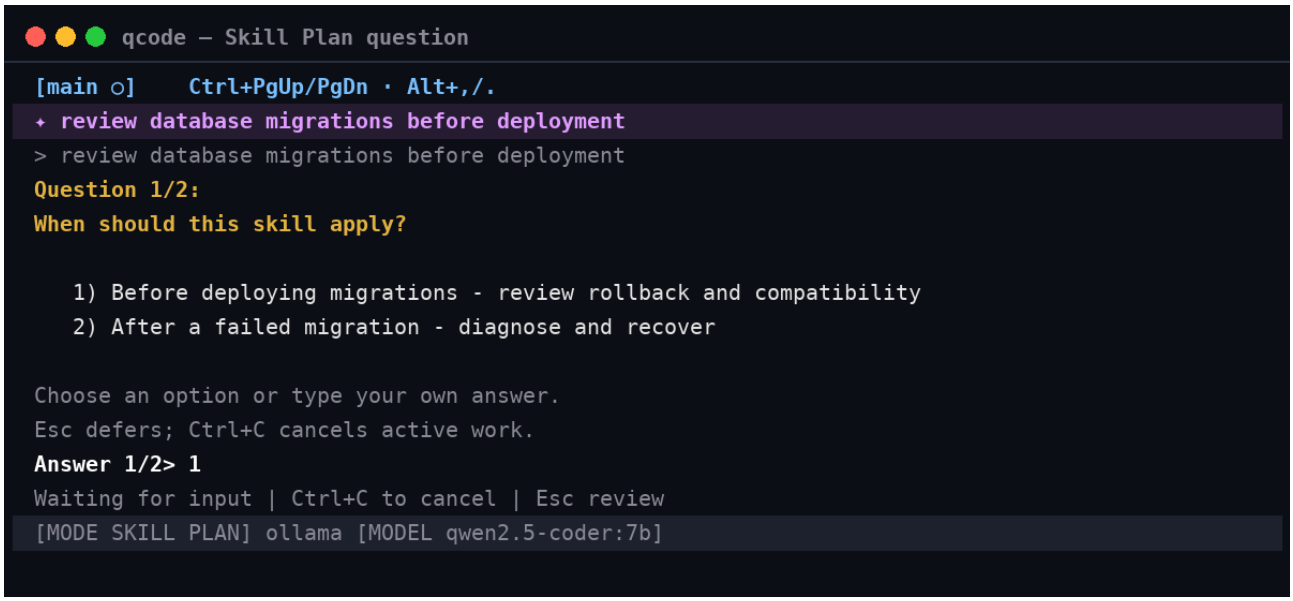


Figure 8: Skill Plan question. Type an option number or your own answer; questions arrive one at a time like Plan and interactive questions.

```
/skillplan review database migrations before deployment
```

- `/skillplan show` opens the latest complete draft in a quoted, scrollable review (Up/Down, PgUp/PgDn, Home/End or g/G; q, Esc, or Ctrl+C closes; the header shows Skill draft | /skillplan create to approve). `/skillplan create` writes exactly the reviewed SKILL.md and leaves Skill Plan mode. `/skillplan off` leaves without writing.
- Creation targets are only ~/.qcode/skills, .agents/skills, and .qcode/skills; the draft name must be 1-64 lowercase letters, digits, hyphens, or underscores and the content must be non-empty valid UTF-8 within 64 KiB. qcode writes only the reviewed SKILL.md (companions must already exist or be created separately) and never overwrites an existing SKILL.md; a name or location conflict stays in Skill Plan mode so you can pick another.
- When the design is coherent, qcode asks whether to Create skill (same as /skillplan create) now or Stay in Skill Plan mode to review it with /skillplan show or request refinements; each complete revision replaces the previous draft. After creation, run `/skill` to refresh discovery and enable the new skill.
- Skill Plan mode advertises only inspection tools (read, list, search, view_image, web_fetch, web_search, skill, plus agent-history helpers) with ask_questions and propose_skill. Shell, write, edit, directory changes or grants, and sub-agent delegation are rejected at the tool boundary. `/new` clears the saved draft and returns the agent to normal mode, like `/plan off` leaves planning.

9. Remembering preferences with /learn

Use `/learn` after a good answer to save a durable preference or reusable procedure. qcode shows a preview and asks `Apply these global learning changes? [y/N]`. Nothing is saved unless you answer `y` or `yes`; Enter, `n`, or Ctrl+C cancels.

Command	What it does for you
/learn	Propose memory from recent conversation.
/learn list	Show saved items and IDs.
/learn forget <id>	Review then delete one item.
/learn compact	Merge duplicates (asks first, backs up).

- Memory is per user on this device and shared across folders. Retrieval ranks learning against the latest prompt (up to 3 items, about 1200 tokens by default).
- Set context_budget = 0 under [learning] to stop retrieving learning while keeping /learn working and saved records intact.
- Keep entries reusable: state the language or condition when they apply.
- Review proposals: secrets are rejected, but you decide what is worth keeping.

10. Tools, working directories, approvals, and web

Choosing tools

Use /tool to filter and toggle the active tab's tools: read, write, edit, list, search, shell, change_directory, view_image, skill, web_fetch, and web_search, plus request_directory_access where offered. Space toggles; Enter applies. Or use /tool <name> on|off, for example /tool shell off. New conversations restore tool defaults. Plan and Skill Plan show only their allowed tools; /interactive controls questions, not /tool.

Working directories

Every agent starts in the launch workspace (--cwd) and keeps its own current directory. Ask for another directory (for example packages/server) and the agent can call change_directory; absolute paths and parent-relative paths such as ../notes.txt are accepted, relative paths resolve against that agent’s directory, and symlinks resolve to their canonical destination. The terminal and browser status WS segment follows the selected tab. Other agents and the qcode process keep their own directories.

- Without the sandbox, read, list, search, and view_image may inspect paths outside the agent’s directory, subject to OS permissions. write and edit still reject paths that lexically leave that directory, but symlinks and shell can reach anything your OS account allows. change_directory can select an existing directory outside the launch workspace; later write/edit confinement follows the newly selected directory.
- The model can call request_directory_access when work needs another directory. qcode asks you to review an editable path, grants it read/write for the session, and clears added grants on /new. The filesystem root cannot be granted. Non-interactive runs deny the request. Switching to an already approved directory needs no prompt; an unapproved destination needs approval before the switch or creation proceeds.
- /resume restores saved grants when paths remain valid under current protections, else warns and falls back to launch. /new clears added grants and resets the active directory to launch.
- For large files, ask for a line range; read supports offsets and limits, and search reports matching line numbers. Name a workspace image in your prompt to inspect it with a vision model (PNG, JPEG, WEBP, GIF, 20 MiB each).

Approvals and untrusted content

Your request and qcode’s built-in instructions set the task. Web pages and snippets, local files and search results, shell output and errors, loaded skills, agent handoffs, saved conversations, and imported context are reference data: they can supply facts but cannot authorize tool calls. When qcode sees an actionable instruction pattern inside untrusted content, it prints a calibrated warning without repeating that content.

- Later calls that can access data, change state, or send data then need your approval for the exact call. In one-shot or other non-interactive use those calls are denied. You can also approve subsequent tool calls for the current agent run; approval does not carry to the next run. Other tool and sandbox restrictions still apply.

- After a prompt-injection warning, activity and trace arguments are redacted; the approval prompt still shows the exact proposed call for review. Run /verbose before the tool runs to see the full sanitized output behind the hidden preview.
- Plan mode is stricter: only inspection tools are advertised and every mutation-capable call is rejected before execution. It never runs shell, requests directories, or delegates to other agents.

Using web access

Web tools read public pages and search results as text. Both start enabled outside sandbox mode and disabled when a sandbox is active, in interactive and one-shot runs. Use `/tool` to change the active tab's selections. The web_search backend setting does not change tool enablement.

- Type `/tool` to toggle `web_fetch` and `web_search` independently, or use `/tool web_fetch off` to disable fetching.
- `/new` and fresh launches restore these defaults. Resume restores saved selections, and helpers created by main inherit its tools; an enabled web tool allows sandbox networking.
- Use them for docs and public references. They do not run page scripts and do not open a browser.
- Limits: 20-second deadline, at most 5 redirects, 2 MiB response cap, at most two concurrent web operations, 64 KiB text cap with truncation marker. Only public destinations are accepted; loopback, private, link-local, and reserved addresses are rejected, including DNS results and redirects. Requests use normal TLS unless --danger-skip-tls-verify is set.
- In sandbox mode, enabling either web tool allows networking for native web tools and sandboxed shell commands (curl, package downloads); disabling both or using /new restores isolation.

Tip

If a page needs login or runs an app, fetch will not see it. Paste the relevant text into qcode instead.

11. Sessions: autosave, resume, and export

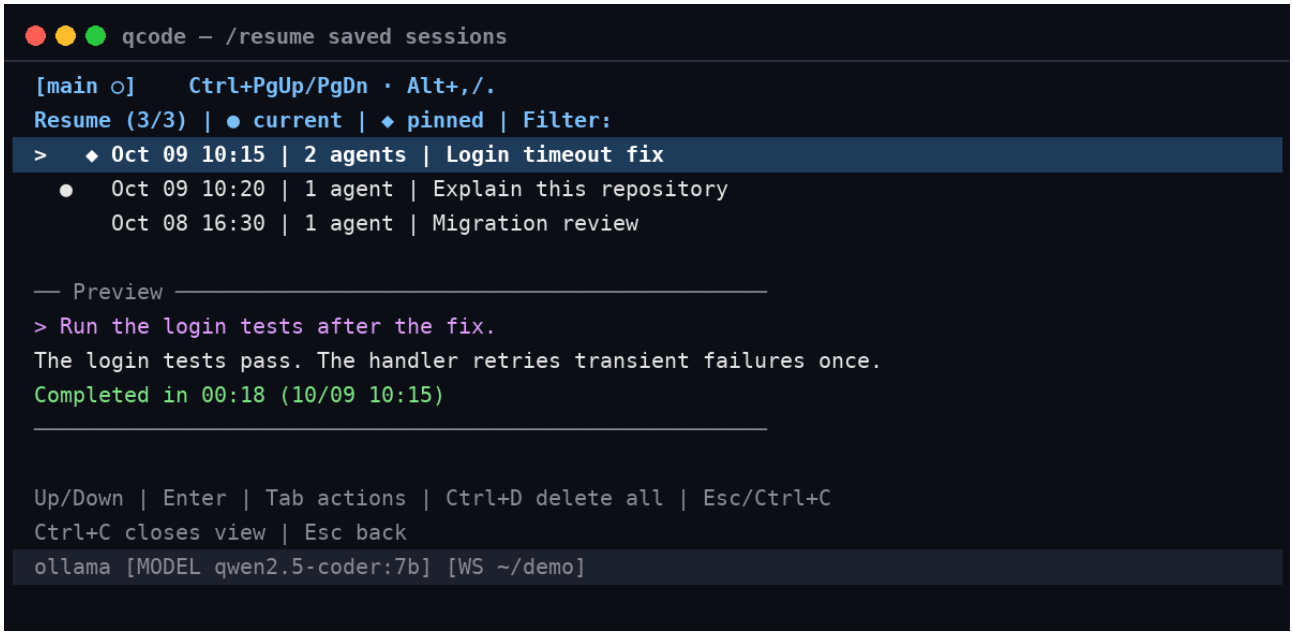


Figure 9: /resume previews the saved active tab. Pinned sessions sort first; Tab opens Rename, Pin/Unpin, and Delete, while Ctrl+D opens Delete all.

- Sessions autosave about every two seconds, after agent work and commands, and on clean exit or session switch. Each launch starts its own session.
- Type /resume to reopen a session for this folder. A separate marker identifies the current session; selecting it keeps it open. Entries show a preview and activity time. Pinned sessions appear first; prompt activity orders each group. Tab opens Rename, Pin/Unpin, and Delete; Ctrl+D opens Delete all from the list. Up/Down chooses an action; Enter applies it. Finish or cancel running work before switching or deleting the current session. Concurrent processes can restore the same session, with the last save winning.
- Ctrl+D opens Delete all with Cancel selected by default. Cancel or Esc returns to the list with its filter and selection intact. It deletes every saved session in this workspace, including sessions hidden by the filter, along with names and pins. Other workspaces are unaffected. If the current session is saved, it starts fresh after the other removals succeed; an unsaved current conversation stays open. Failures refresh the list and retain the current conversation.
- Delete requires confirmation with Cancel selected by default. Enter applies the choice; Cancel or Esc returns to actions; Ctrl+C closes the picker. Deleting another session keeps the filter and selects the next match, or the previous match if it was last. Deleting the current session clears all tabs, drafts, history, plans, usage, work history, and added directory grants, then starts fresh with the main agent's current provider, model, thinking, runtime settings, skills, and tool selections. Deletion removes the saved snapshot, custom name, and pin; another process's later autosave may recreate it.

- Resume restores tabs, models, the pinned latest prompt, conversation and styled output (including blank lines in the preview), diffs and expandable diff data, drafts, reading positions, tool and skill settings, context accounting, and token totals. Provider credentials come from current configuration. Existing directory grants are restored when paths remain valid; discarded grants produce a notice.
- Sessions containing only slash commands, unsent drafts, settings, or idle agent tabs are not saved; saving starts with user conversation or recorded agent work, including queued prompts. Previously saved empty sessions stay available for manual deletion. Demo and one-shot runs are never saved. `/new` resets only the current tab inside its session.
- After a crash, start qcode in the same folder and use /resume. The last good checkpoint marks unfinished work interrupted; pending tasks and steering are never resubmitted automatically. Inspect current files before continuing, since completed tool changes are not rolled back.
- Sessions live outside your project: ~/.local/state/qcode/sessions on Linux (or $XDG_STATE_HOME), ~/Library/Application Support on macOS, %AppData% on Windows. They keep private file permissions.

Local privacy and redaction

Local text filtering is on by default. Detected credentials, private keys, payment-card numbers, and labelled card security codes or expiry dates appear as [REDACTED] in output, thinking, tool previews, saved sessions, exports, and browser views. Live editable input stays visible while you compose it; submitted history and saved draft copies are filtered.

When you resume an older session, qcode filters and rewrites it before showing the restored conversation. If that rewrite fails, resume stops. Redacted saved text cannot be recovered, so the resumed conversation may need missing context supplied again.

Filtering applies to local text, not requests sent to a model provider, shell commands, workspace file writes, or image contents. It covers five sinks (terminal, persistence, exports, json_events, remote); each can be toggled under [redaction]. Detection can miss unfamiliar formats. Use [redaction] settings for custom patterns, fields, and paths; see docs/privacy.md and docs/configuration.md for coverage and options.

12. Controlling qcode from a browser

Type /remote to control the same session from a phone or browser. Choose Pure Web for a trusted LAN, Tailscale for your tailnet, or Pure Web (No auth, danger!) for open control by anyone with the URL; reserve no-auth for open, short-lived trusted-network demos. Pure Web is plain HTTP on your LAN: use only networks you trust.

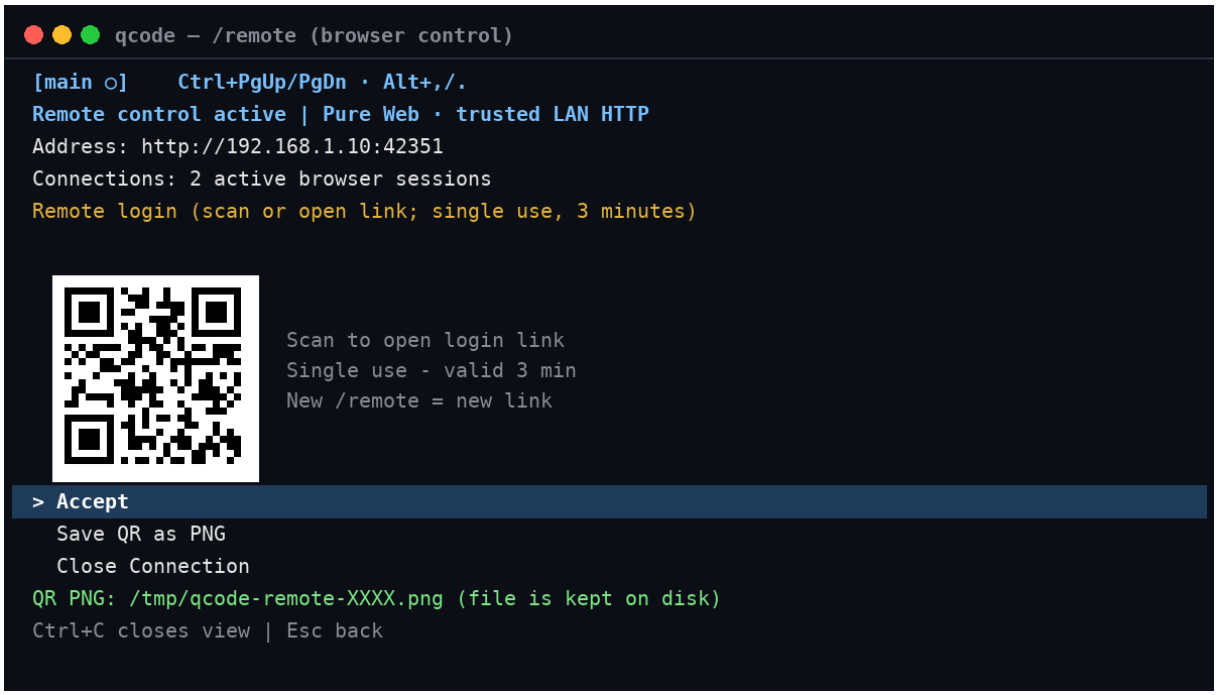


Figure 10: /remote screen. Scan or open the link, then Accept. Save QR as PNG if it does not fit.

1. Type `/remote` and choose how to connect. For multiple networks, pick the interface to share.
2. Scan or open the link on the other device. Links are single-use and last 3 minutes. A new link replaces an unused one; existing browsers stay connected.
3. If the code does not fit, choose `Save QR as PNG` and open the shown file path (in WSL, convert it with `wslpath -w <path>`). The file is kept on disk.
4. Choose Accept to keep the connection open. Run `/remote` again for one more device; existing browsers stay connected.
5. To stop all browsers, choose Close Connection and confirm, or quit qcode. Used and expired links tell you to run `/remote` again.
6. If more than one active LAN IPv4 interface exists, qcode asks which one to expose (name, address, and CIDR); with one eligible interface it uses that address directly.
7. Tailscale binds a loopback listener and proxies it with a foreground tailscale serve process. It tries HTTPS port 443 first, then 8443, then 10000-10031; the selected port appears in the browser URL and QR link. Tailnet access rules must allow that port. Browsers must belong to the same tailnet.

- Browser Enter and Steer update the busy task; Queue submits a separate task. Tab navigates controls normally.
- Remove cancels pending input; Cancel active work stops the running task. Rejected submissions keep your draft.

- The selected agent's latest received prompt stays below the browser header, up to three lines, while the transcript scrolls. It updates only when a task starts or steering is delivered.
- Beginning and Latest navigate transcript boundaries. Home/End do the same outside text fields, command panels, and the expanded pending list; text fields retain their normal editing keys.
- Reloading the same browser tab resumes access; a fresh browser needs a fresh link. /export downloads the same HTML as in the terminal.
- Use only on networks you trust. Pure Web is plain HTTP on your LAN.

13. Settings you actually change

Most daily choices live in the terminal (`/model`, `/maxsteps`, `/statusline`, `/theme`, `/tool`). Use a config file only for defaults you always want.

Terminal colors and workspace visibility

Use /theme to preview terminal palettes with Up/Down. Enter applies and saves the choice; Esc or Ctrl+C cancels. Default (Auto) follows your terminal. The 22 dark and light choices include Catppuccin, Dracula/Alucard, Gruvbox, Solarized, Nord, Tokyo Night, One, Rosé Pine, Everforest, Kanagawa, and Ayu. The setting is terminal-wide and saves from any agent tab. The browser's pinned prompt follows the theme's accent color and background.

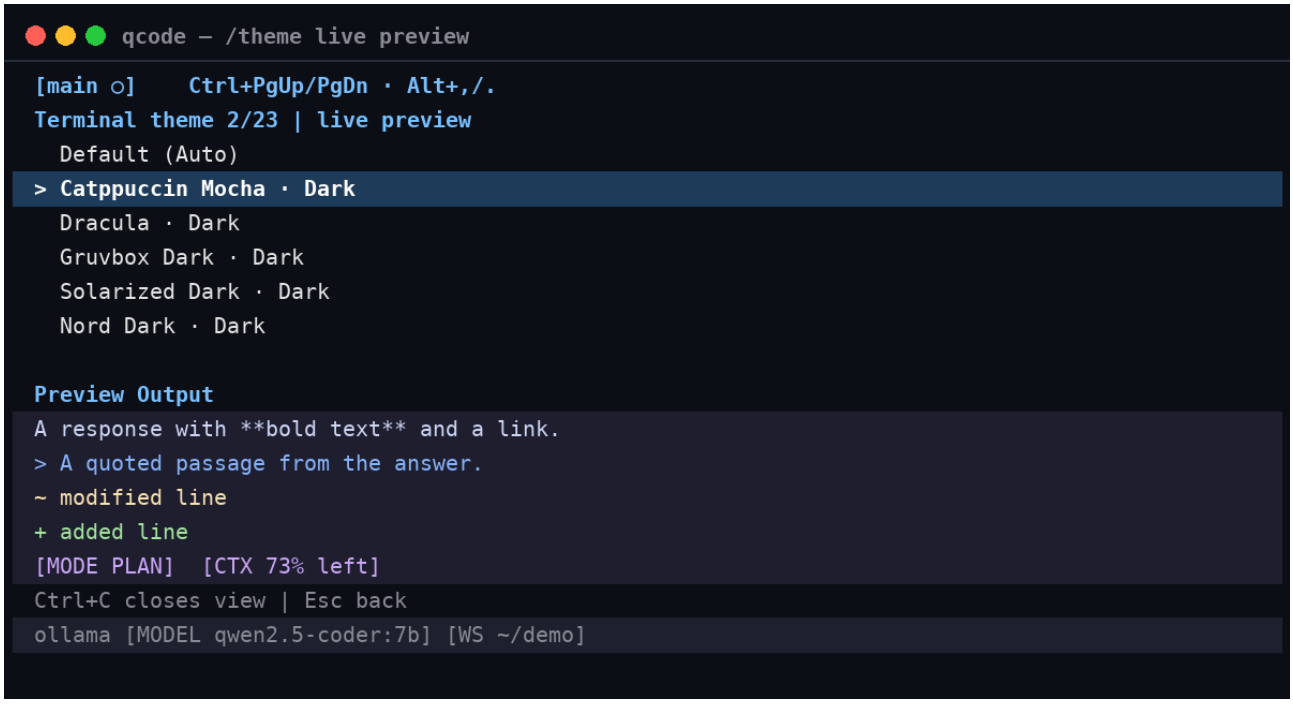


Figure 11: /theme previews colors for responses, diffs, and status. Enter applies and saves; Esc or Ctrl+C leaves the current choice.

WS uses available status bar space and keeps the final two complete folders when possible, using a second row as needed. Home-relative paths retain ~/; omitted parents appear as an ellipsis folder, for example ~/.../work/foo. If the final folder name must be clipped, its ending gets an ellipsis: ~/workspace/this-folder-is... (the final dots shorten the name).

TOK shows provider-reported session input/output counts. The combined total uses a sigma marker, or T: in ASCII mode, whenever it fits one or two status rows. If usage is missing, unknown or a question mark indicates incomplete counts. Unknown means unreported, not zero; partial subtotals are not bills; cached tokens occupy context and count inside input totals; mixed-provider totals are token sums only. Use /usage for the detailed read-only view (input/output/total, cache-read/write sums with reported-request coverage, missing completions, legacy uncertainty, and cached input share). /new resets the active agent's totals; changing models keeps them.

Open /statusline, move with Up/Down, toggle with Space, and apply with Enter. Or type /statusline tok off, /statusline show, or /statusline reset. Changes on main save statusline_hidden in your user

config while preserving comments and unrelated tables, including redaction and skills settings.

```
provider = "ollama"
model = "qwen2.5-coder:7b"
max_steps = 32
theme = "catppuccin-mocha"
sandbox = true
```

- Copy `config.toml.example` to your user config location; missing files are ignored. Priority, highest first: file beside the program, user config (~/.local/etc/qcode on Linux, ~/Library/Application Support on macOS, %AppData% on Windows), system config (/usr/local/etc/qcode), then etc/ beside the program.
- Command flags beat environment variables. Both beat config files. Config files beat built-ins. `--api-key` beats config and environment keys.
- Prefer environment variables for keys: QCODE_API_KEY or OPENAI_API_KEY. Keep key files private (mode 0600).
- Common knobs: provider/model/thinking, agent_timeout like "5m", interactive true/false, statusline_hidden, web_search backend, learning context_budget, skills paths, autoload_paths, selected/selected_always, [[agents.allow_models]] roster, [providers.<name>] profiles, [chatgpt] oauth_path, redaction sinks/patterns, context_window and auto_compact_threshold.
- Restart after changing the web backend. /model, /maxsteps, /statusline, /skill, and /agent allow-models on main save automatically; worker-tab changes to these settings stay within the session.

Working directories under the sandbox

In sandbox mode the same per-agent directory rules apply, but confinement is stronger: each shell call gets a fresh namespace with no network by default, a hidden home and runtime, a private temporary directory, a read-only root, and read/write mounts for approved directories. qcode itself stays outside the sandbox so it can load configuration and contact the provider; API keys and other environment secrets are removed from shell-tool environments, and every qcode configuration file inspected at startup is hidden from model tools. Built-in file tools use kernel-assisted confinement to prevent symlink escapes. Resume restores saved tool selections, so an enabled web tool also allows networking in those sandbox sessions.

Sandbox and trusted commands

On Linux, use --sandbox with bubblewrap installed and check the startup notice to confirm isolation is active. Tools can change only approved directories; home is hidden. Fresh sandbox sessions start with both web tools off and networking blocked. Enabling either through /tool allows networking for native web tools and sandboxed shell commands. Disabling both or using /new blocks it again. If sandbox setup fails, the terminal offers to continue without it or leave; one-shot runs warn and continue without it. Continuing without a sandbox starts with both web tools enabled.

```
qcode --sandbox --sandbox-command-path ~/.local/bin

# Persistent defaults in config.toml:
sandbox = true
sandbox_command_paths = ["~/.local/bin", "~/go/bin"]
```

Configured sandbox = true and --sandbox use the same defaults; --sandbox=false overrides configuration, including web-tool defaults. Trusted command directories are mounted read-only at their original paths (only empty ancestors are created under the hidden home) and kept across /new, never read/write, and never prompt. Missing, invalid, duplicate, root, or protected directories produce warnings. Additional writable directories require your approval and are cleared by /new. Temporary shell files do not survive separate sandbox calls; use workspace files for results you need later.

TLS, keys, and provider routing

Prefer QCODE_API_KEY or OPENAI_API_KEY for credentials; --api-key beats both environment variables and the config file. danger_skip_tls_verify = true (or --danger-skip-tls-verify) disables certificate and hostname verification for provider and native web requests and supplies insecure-TLS settings to Git, Node.js, npm, and compatible Python shell clients; other shell programs may need their own insecure-TLS flag. When a flag or environment variable selects a different provider than the configured one, the configured model, thinking, base_url, api_key, and context_window are not inherited.

Context capacity

CTX shows remaining capacity, with ~ for an estimate or unknown when no limit is available. Use --context-window 32768 or context_window = 32768 for an unlisted model; this sets terminal-agent accounting and compaction capacity without changing the provider's allocation. Switching models clears the override. --disable-auto-compact disables automatic summaries in terminal sessions; /compact remains available. One-shot runs use provider capacity and the built-in 80% threshold.

14. Troubleshooting and tips

Command	What it does for you
No colors?	Unset NO_COLOR to enable colors. Set NO_COLOR=1 to disable them; QCODE_ASCII=1 uses ASCII glyphs while retaining color.
Narrow window?	The path shortens first; the bar wraps to a second left-aligned line. Low-priority parts drop only if two lines still overflow. Hide more with /statusline.
Slow session?	Try /compact, then /new for a fresh tab context.
Need detail?	Try /verbose. For step limits use /maxsteps N.
Lost output?	Scroll with PageUp/PageDown. /history finds old answers. /export saves HTML.
Resume failed?	Read the inline error; finish or cancel local running agents before switching. Fix snapshot, redaction-rewrite, or save errors, then retry. Other processes do not lock the session.
Waiting a long time?	Read the model operation and elapsed timer above the status bar. Use Ctrl+C from the composer to cancel; tool activity appears in the transcript.
Text is redacted?	Detected sensitive text is replaced locally by [REDACTED]. Saved redactions are permanent; review docs/privacy.md for scope.
Remote blocked?	On Linux run once: sudo tailscale set --operator=$USER. Then retry /remote.
Approval prompt?	Review the exact proposed call. Untrusted page, file, or shell text never authorizes it. Approve only the current run when you understand the action; one-shot runs deny such calls.
Paste collapsed?	Long pastes show [paste N characters]. Left/Right jumps the block; Backspace/Delete removes it whole. Short pastes stay inline. Pasting never submits.
Model switch warns?	A cross-backend model switch warns before applying. Same-backend switches apply silently. Unsupported thinking levels fail before any request.
Directory denied?	Approve request_directory_access or change_directory to an approved directory. The root can never be granted. /new clears added grants and resets to launch.
Compacted context?	Run /compact show to inspect system + summary + recent tail after any compaction.

Tip

Good defaults for most users: local model for drafts, stronger hosted model for final edits, Plan mode for risky changes, one agent per independent job.

15. Cheat sheet (copy this page)

Command	What it does for you
/help	Help for all or one command.
/bash <cmd>	Run a shell command.
/model [id] [thinking]	Switch model by name only (+ thinking).
/agent ...	Create, list, switch, rename, cancel, close.
/plan ...	Plan, show, act, off.
/skill	Enable instruction bundles.
/skillplan ...	Draft, show, create, or leave skill design.
/interactive	Let normal work ask up to 3 questions per prompt.
/learn ...	Save/list/forget/compact preferences.
/memo [text]	Save or manage global scratchpad memos.
/tool	Choose file, shell, image, skill, web, directory tools.
/usage	Detailed token usage for the active agent.
/resume, /new	Reopen session / reset tab.
/history	Find and re-read answers.
/diff [N]	Expand saved diff N (to 200 lines).
/compact [show]	Summarize context; show inspects it.
/verbose	Show/hide detailed traces.
/export	Save pretty or raw HTML.
/remote	Browser control via QR link.
/statusline	Choose status bar parts.
/theme	Preview and save terminal colors.
/maxsteps	Show/set step limit.
/clear	Clear view and redraw header.
/quit, /exit	Save session and exit.

One-shot recipes:

```
qcode --model qwen2.5-coder:7b "explain this repository"
qcode --provider openai --model gpt-5 "summarize the diff"
qcode --demo "show me how qcode works"
```

About this manual

This manual teaches use, not construction. It omits source layout, network internals, and developer extension points on purpose. Approvals, directories, and redaction are covered because you operate them. For command details, type `/help` inside qcode. Generated November 07, 2026 as version dev. Screenshots are illustrative examples rendered for print clarity.

Chapters in this manual

- How to use this manual
- 1. Getting started
- 2. Touring the terminal
- 3. Asking, queueing, and history
- Use cases: understand and fix; plan and divide work; reuse and continue
- 4. Slash commands at a glance
- 5. Choosing models and providers
- 6. Working with multiple agents
- 7. Planning before coding
- 8. Skills and SkillPlan
- 9. Remembering preferences with /learn
- 10. Tools, working directories, approvals, and web
- 11. Sessions: autosave, resume, and export
- 12. Controlling qcode from a browser
- 13. Settings you actually change
- 14. Troubleshooting and tips
- 15. Cheat sheet